FRANKFURT UNIVERSITY OF APPLIED SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Project Applications Report

Object Oriented programming Java module

Winter Semester 2023/2024

Supervisor: Prof. Dr. Doina Logofătu
Student 1: Cong Nguyen Le - 1519392
Student 2: Thien Huong Duong - 1527403
Student 3: Minh Anh Nguyen - 1525087

January 31, 2024

Content

1 Abstract

In this report, we present the development and analysis of a Java-based application that simulates the popular board game "Da Ist Der Wurm Drin"[1]. This project was undertaken within the Java Object-Oriented Programming (OOP) module at Frankfurt University of Applied Science, under the guidance of Prof. Dr. Doina Logofătu.

2 Teamwork

2.1 Teamwork

Our team comprised three individuals, each with a specific role below:

Project manager	Cong Nguyen Le, Thien Huong Duong
Developer	all team members
Tester	Thien Huong Duong, Minh Anh Nguyen
Game Designer	Thien Huong Duong, Cong Nguyen Le
Code Documentation	all team members

Table 1: Team member roles

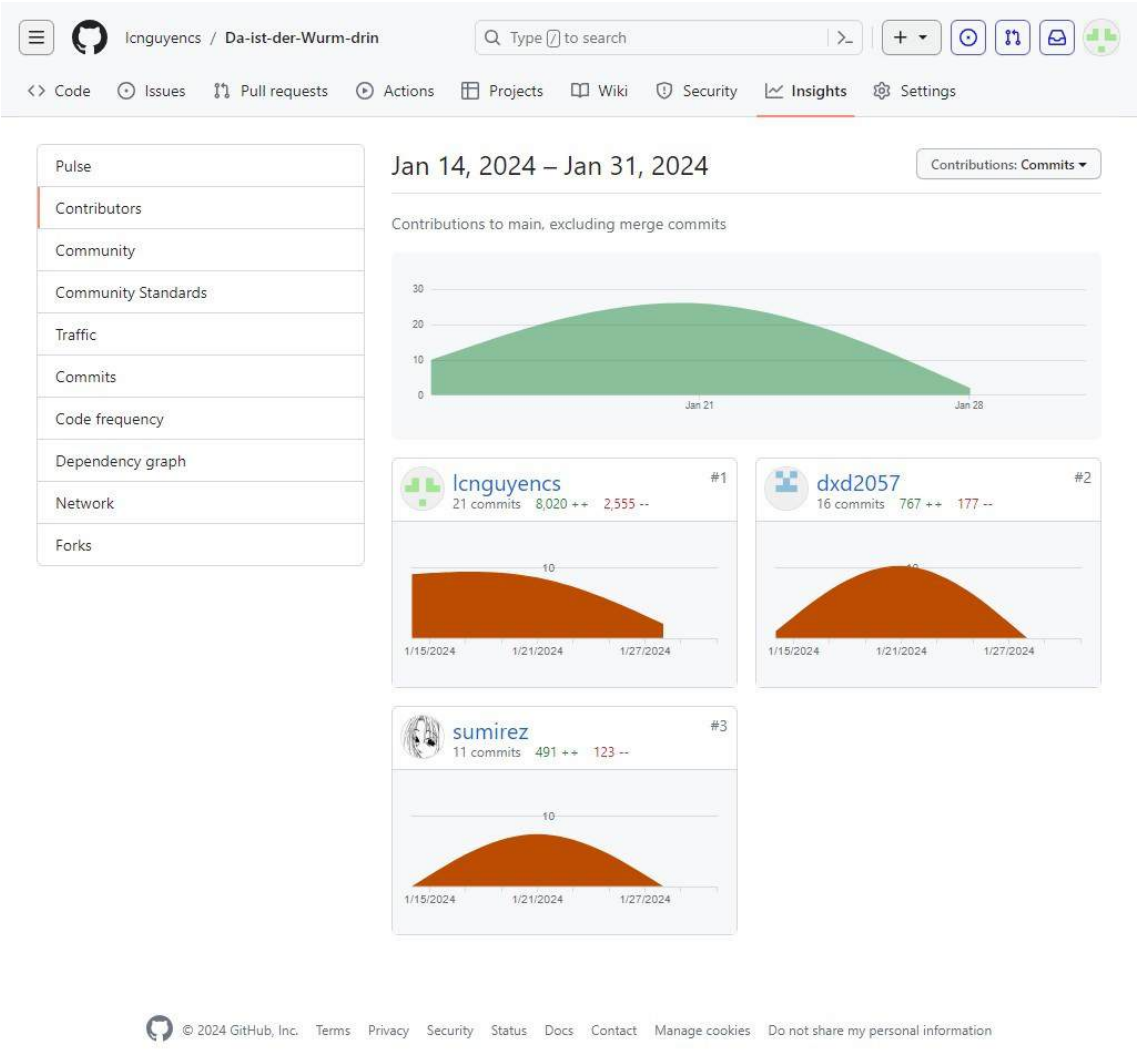


Figure 1: contribution of team members

2.2 Work Structure

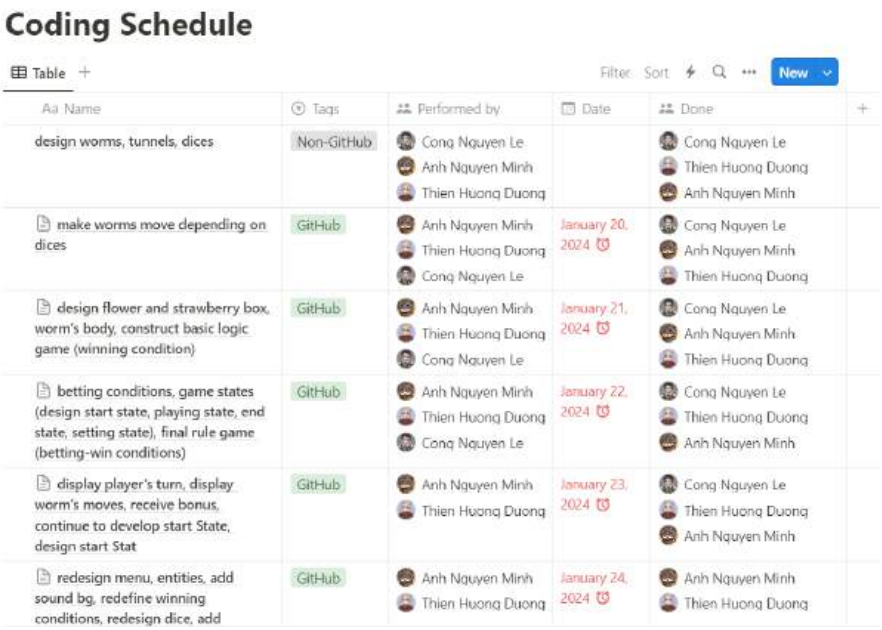
Our team employed a wide array of tools and methodologies to ensure an effective and streamlined workflow. These included communication tools, decision-making processes, repository management, and bug tracking systems. The aim was to foster collaboration, maintain high standards of code quality, and ensure efficient problem resolution.

For communication, we leveraged daily updates and discussions through **Whatsapp**. This platform allowed us to stay connected and updated at all times. In addition, we conducted weekly in-person meetings to align our efforts and make crucial decisions together. This face-to-face interaction helped us to better understand each other's perspectives and come to consensus on important matters.

When it came to making decisions, we adopted a collaborative approach during team meetings. Each member had a voice, and we considered different viewpoints before reaching a conclusion. However, the ultimate decision rested with the Project Manager. This structure ensured that there was a clear chain of command while still allowing for input from all team members.

In terms of managing our codebase, we utilized **GitHub**[2]. We implemented feature branching and pull requests to ensure code quality and facilitate peer reviews. Feature branching allowed us to work on new features without affecting the main codebase, while pull requests provided an opportunity for others to review and approve the changes before they were merged into the main branch. This system helped us maintain a clean and manageable codebase.

Tracking bugs was another crucial aspect of our workflow. We used **Notion** for this purpose, which allowed us to assign issues to responsible team members for resolution. By doing so, we could keep track of who was working on what, when it was due, and its current status. This helped us to efficiently manage our time and resources, and ensure that all issues were addressed promptly.



Coding Schedule

Table + Filter Sort New

Name	Tags	Performed by	Date	Done
design worms, tunnels, dices	Non-GitHub	Cong Nguyen Le, Anh Nguyen Minh, Thien Huong Duong		Cong Nguyen Le, Thien Huong Duong, Anh Nguyen Minh
make worms move depending on dices	GitHub	Anh Nguyen Minh, Thien Huong Duong, Cong Nguyen Le	January 20, 2024	Cong Nguyen Le, Anh Nguyen Minh, Thien Huong Duong
design flower and strawberry box, worm's body, construct basic logic game (winning condition)	GitHub	Anh Nguyen Minh, Thien Huong Duong, Cong Nguyen Le	January 21, 2024	Cong Nguyen Le, Anh Nguyen Minh, Thien Huong Duong
betting conditions, game states (design start state, playing state, end state, setting state), final rule game (betting-win conditions)	GitHub	Anh Nguyen Minh, Thien Huong Duong, Cong Nguyen Le	January 22, 2024	Cong Nguyen Le, Thien Huong Duong, Anh Nguyen Minh
display player's turn, display worm's moves, receive bonus, continue to develop start State, design start Stat	GitHub	Anh Nguyen Minh, Thien Huong Duong	January 23, 2024	Cong Nguyen Le, Thien Huong Duong, Anh Nguyen Minh
redesign menu, entities, add sound bg, redefine winning conditions, redesign dice, add	GitHub	Anh Nguyen Minh, Thien Huong Duong	January 24, 2024	Anh Nguyen Minh, Thien Huong Duong

Figure 2: Our Notion schedule for project

As for the software applications we used during this project, we incorporated several tools. For instance, we used **LibGDX**[3], a Java game development framework, to build our game. Android Studio served as our Integrated Development Environment (IDE) to run the game project. **VSCode** was our tool of choice for bug tracking and code documentation, while GitHub controlled our repository. Notion was instrumental in dividing tasks among team members, **Tiled**[4] was used for designing maps and entities, and **Paint 3D** were used to adjust image dimensions. Lastly, **Adobe Firefly**[5] was used to generate images such as MenuState background when the game start.

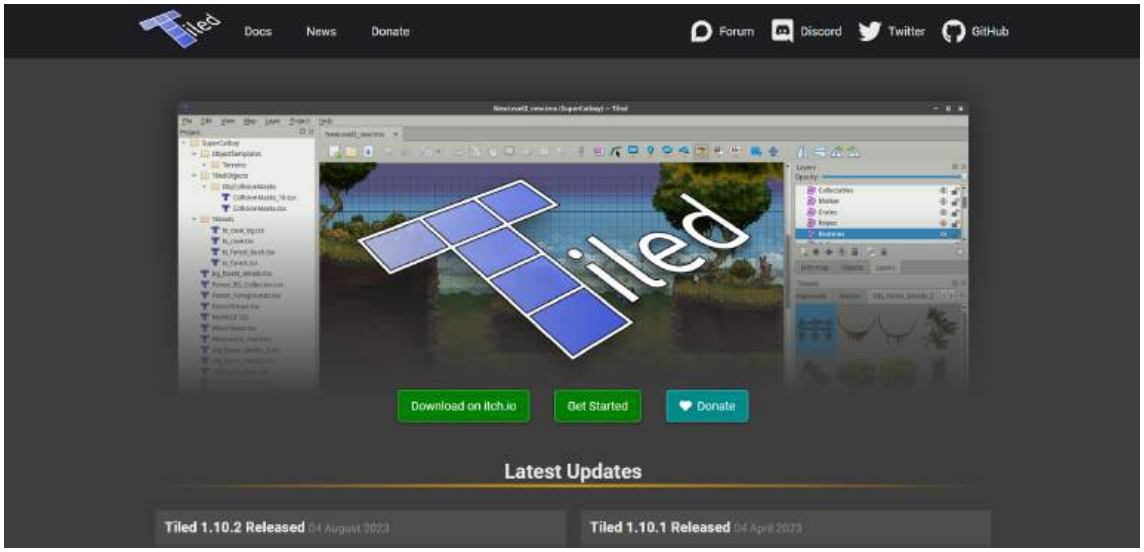


Figure 3: Tiled software homepage

3 Ideas

In the realm of game development, the initial phase is often marked by a flurry of creative brainstorming. Our team was no exception, as we embarked on a methodical journey of idea generation and problem-solving. This detailed account aims to provide a transparent view into our brainstorming process, highlighting the myriad of ideas we considered, the paths we chose, and the innovative solutions we crafted. Our narrative captures the essence of collaborative creativity, demonstrating how diverse inputs can coalesce into a singular, unified vision for a game. From conceptualization to execution, every step in our process was a testament to the power of collective thinking and strategic planning.

3.1 Choosing a Framework

The selection of a suitable framework is a critical decision in game development, and our team approached this task with thorough research and deliberation. After a week of intensive exploration and analysis, we chose the **libGDX** framework over Java Swing. This decision was primarily influenced by libGDX's superior capabilities for game development. It offered enhanced graphics performance, which was vital for creating visually appealing gameplay. Additionally, its cross-platform support opened avenues for broader audience reach. Another compelling factor was the active community surrounding libGDX, offering valuable resources and support. This combination of features made libGDX an ideal choice for our project, laying a solid technical foundation for our game.

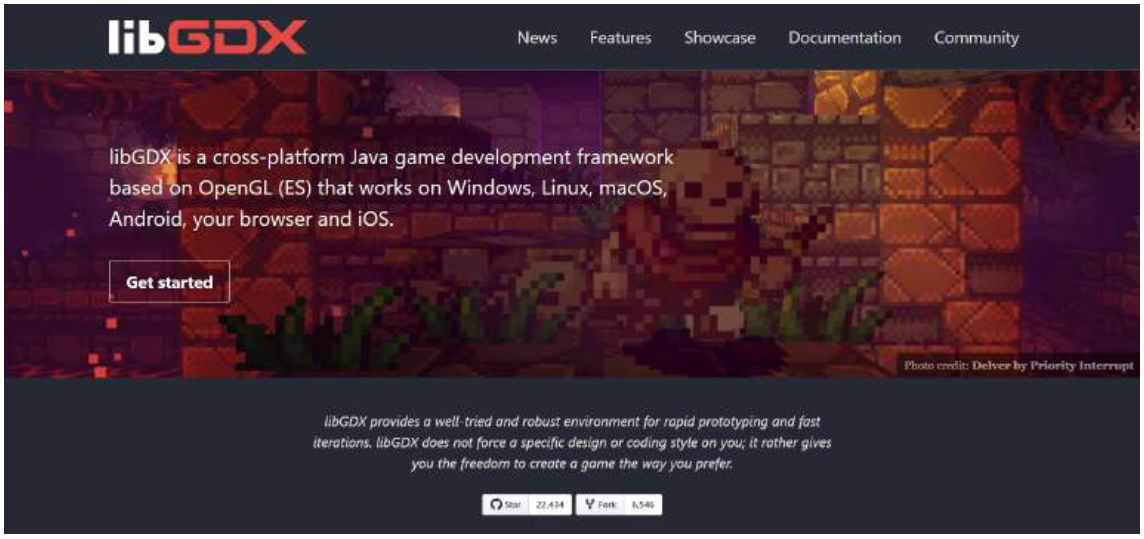


Figure 4: libGDX platforms homepage

3.2 Learning and Implementation

Embarking on the journey with libGDX required us to invest significant time in learning its intricacies. Mastering this framework was essential for leveraging its full potential, ensuring that we could build a robust and engaging game. We dedicated ourselves to understanding the nuances

of libGDX, from its core functionalities to more advanced features. This phase of learning was a crucial investment, setting the stage for a more efficient and effective development process. It enabled us to build a strong foundation for our game, ensuring that the technical aspects were handled with expertise and precision.

The inaugural feature we developed was the **worms' movement**, a core gameplay mechanic based on dice rolls. This was a strategic starting point, as it established the fundamental interaction upon which the rest of the game was built. Our focus on this element underscored its importance in the overall gaming experience. The development of this feature involved intricate coding and careful testing, ensuring that it functioned seamlessly within the game's environment. This initial step was pivotal, as it not only set the technical framework for subsequent features but also provided us with valuable insights into the game development process.

3.3 Game Mechanics and Features

In our quest to create a dynamic and engaging gaming experience, we introduced **tunnels** towards the end of the game. This addition was designed to infuse an element of surprise and excitement into the gameplay. The tunnels served as strategic hiding spots for the worms, adding a layer of unpredictability that kept players on their toes. The implementation of this feature involved careful planning and design, ensuring that it integrated seamlessly with the existing game mechanics while enhancing the overall appeal.

Following the integration of tunnels, we implemented a **betting set** feature. This allowed players to wager on certain outcomes, introducing a strategic dimension to the game. The betting system was designed to be intuitive and engaging, encouraging players to think critically about their choices. However, we encountered a challenge with the betting system: players were initially unable to modify their bets once placed. To resolve this, we developed an array system to track the **status of each bet**. This solution enabled dynamic updating of bets, granting players greater flexibility and control over their strategic decisions.

3.4 Enhancements and Fine-Tuning

One of the key elements we focused on was the **winning condition**, which was intricately tied to the worms' movements. The complexity of this feature stemmed from its dependence on the screen's dimensions, requiring us to strike a delicate balance. Our goal was to ensure fairness and playability, which necessitated meticulous calibration and testing. This process involved fine-tuning the algorithms that determined the winning conditions, ensuring they were responsive to the game's dynamic environment. Additionally, we placed emphasis on **displaying the current player's turn** on the screen. This feature was crucial for maintaining player engagement and ensuring a smooth game flow. It kept players informed of the game's progression and allowed them to strategize effectively.

To further enhance the gaming experience, we implemented a notification system. This system was designed to inform players about the **bonus moves received by worms due to successful betting**. It was a critical addition that helped players understand the impact of their betting strategies on the gameplay. This feature was not only informative but also added an element of excitement, as players could see the direct consequences of their decisions. The implementation required careful integration with the game's existing mechanics, ensuring that the notifications were timely, relevant, and did not disrupt the flow of the game.

3.5 Aesthetic and Audio Enhancements

A pivotal aspect of game design is the creation of an immersive environment, and the **Menu State background** played a significant role in this regard. We encountered a challenge with the original poster's dimensions, which did not align with our rectangular game design. To overcome this, we explored the use of AI tools for image generation, allowing us to create a background that was not only visually appealing but also fit the game's aesthetic requirements. This endeavor involved learning new techniques and experimenting with different styles, ultimately leading to a background that enhanced the game's visual appeal.

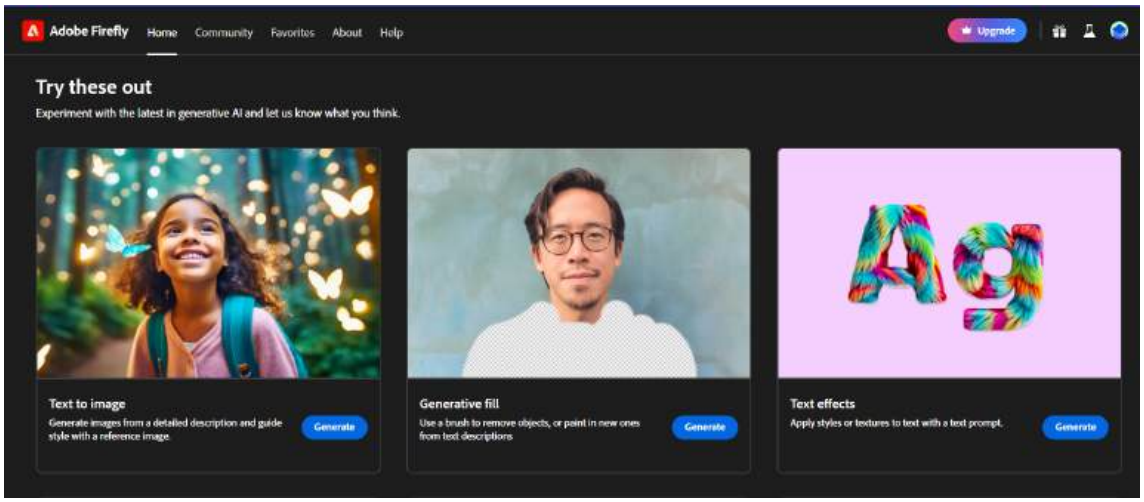


Figure 5: Adobe Firefly for background design

In our quest to enrich the game's ambiance, we incorporated **8-bit sounds**[6] for various elements such as background music, dice rolls, and the winner announcement. This choice was driven by our desire to create a retro feel, resonating with the nostalgic charm of classic games. The selection and integration of these sounds were meticulous, ensuring they complemented the game's theme and enhanced the overall experience. Furthermore, we faced a design challenge with the **buttons**, particularly in scaling the text to fit appropriately. To address this, we developed a mathematical formula that enabled us to center the text within the buttons, regardless of its length. This solution not only improved the aesthetics but also enhanced usability.

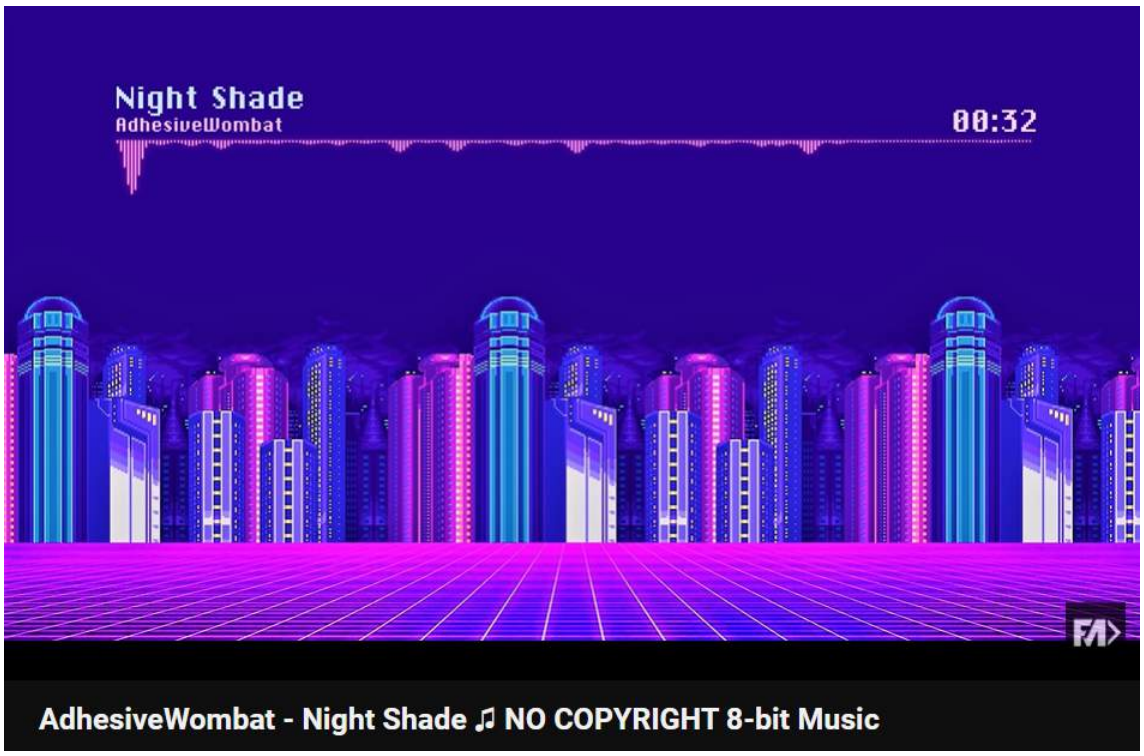


Figure 6: 8-bits sounds for background music

3.6 Font and Sound Integration

The choice of font plays a subtle yet significant role in defining a game's character. In our game, we transitioned from the standard Arial font to **KOMTMT**[7], a decision that was driven by the need to align the text style with the game's overall aesthetic. This change added a layer of visual consistency and thematic coherence, contributing to a more immersive experience. Additionally, we encountered a technical challenge with the sound integration. Specifically, the winner sound would overlap with the background music when a player started a new game. To resolve this, we implemented a mechanism to **pause the winner sound** before continuing with the background music, ensuring a seamless audio experience.

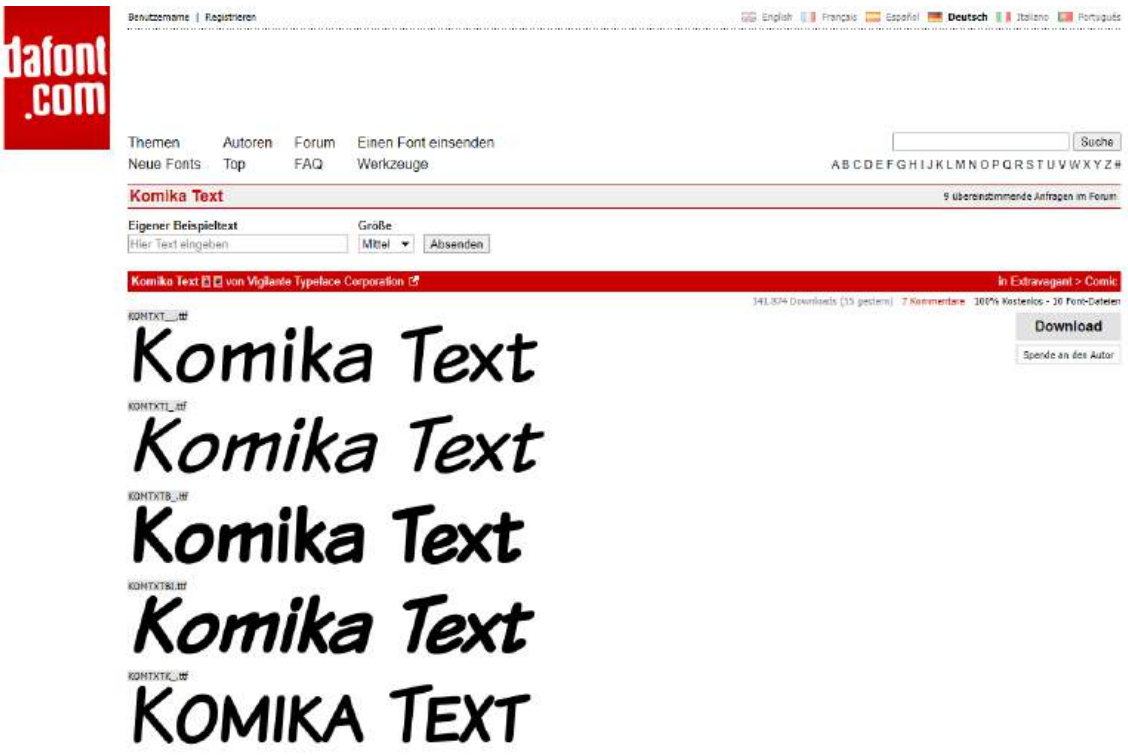


Figure 7: KOMTMT font for display game notification

3.7 Architectural Improvements

An effective game architecture is crucial for smooth navigation and gameplay. We faced a notable challenge in transitioning between the **MenuState** and the **SettingState**. To address this, we introduced a higher-level **GameStateManager** class. This class utilized a Stack data structure to manage state transitions effectively, streamlining the navigation process. This approach was later extended to facilitate transitions from the **EndState** to the **PlayState**. The development and implementation of the GameStateManager class underscored the value of our brainstorming and problem-solving process, demonstrating how architectural improvements can significantly enhance the user experience.

4 Introduction

4.1 General Topic

The game is called "**Da ist der Wurm drin**" which can be roughly translated into The worm is in there. It was developed by Carmen Kleinert and won the Children's Game of the Year 2011 award.

Figure 8: Da ist der Wurm drin game

Each player will choose 1 out of 4 available worms: "**Little Gritty, Stripy Toni, Rudy Red, Lady Silver**"

Figure 9: Avaliable Worm to choose from

The game's objective is to guide your chosen worm through the garden, burrowing it into the soil and being the first to stretch its head out of the soil at the compost heap. Players will move their worm using a color-coded dice and add the color pieces into their worm body. A unique aspect of the game is that its player must estimate and guess which worm will be the fastest and if they guess correctly, they can feed their worm with daisies and strawberries to make it burrow faster. A single game of "Da ist der Wurm drin" took around 10 to 15 minutes.

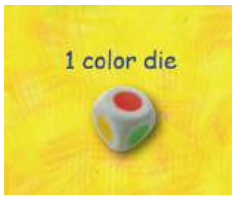


Figure 10: A preview of the color die

Target Audience: The game is designed for 2 to 4 players and both adults and kids who are worm-loving borrowers can join in for the fun, however, the game recommends players aged 4 years and up for the best experience.

Components: The game comes with 1 large game board and 1 small game board with 60 worm sections (in 6 different colors and lengths), and 1 color die. In addition to the worm section, there are 4 worm heads, 4 strawberries, and 4 daises.

4.2 Problems with the game concept

In the development and review of our game, designed with a younger audience in mind, we have identified several critical issues that could impact the fairness, perception, and overall enjoyment of the game. These problems, if unaddressed, might not only detract from the gaming experience but also raise concerns among players and guardians alike. Below, we detail these challenges and their potential implications on the game's dynamics and player engagement.

- One significant issue arises from the rule that allows the youngest player to start the game, which inadvertently introduces an element of unfairness. This rule, while seemingly innocuous, can disadvantage other players right from the outset based on an uncontrollable factor—their age. The game's reliance on the positioning and timing of moves (such as reaching specific spots for winning or making crucial guesses) exacerbates this issue. Without considering the inherent imbalance caused by different starting points, the game may

inadvertently penalize players not only for their strategic decisions but also for factors beyond their control. This problem highlights the need for a more equitable approach to determining player order and game mechanics that do not disproportionately favor or disadvantage participants based on arbitrary criteria.

- Additionally, the game's unique system of guessing, which involves wagering items of value (such as a flower or a strawberry) to potentially increase a worm's length, raises concerns regarding its appropriateness for a children's game. This mechanic, while designed to add excitement and a layer of strategic decision-making, bears a resemblance to gambling practices. Players must risk something of value on the chance of a favorable outcome, a concept that may be viewed as contentious, especially considering the target demographic. The perception of gambling, even in a simplified and seemingly harmless form, necessitates careful consideration and possibly the redesign of this mechanic to ensure it aligns with the game's educational and entertainment goals without introducing controversial elements.

- A further complication is the absence of mechanisms for players who fall behind to catch up or alter the game's trajectory significantly. The lack of catch-up or wildcard rules means that once a player finds themselves at a disadvantage, the likelihood of recovering or overtaking others diminishes as the game progresses. This design flaw can lead to decreased engagement, frustration, and a sense of discouragement among players who perceive their chances of success as minimal. Ensuring that all players remain competitively viable throughout the game is essential for maintaining interest, motivation, and a positive gaming experience for everyone involved.

Addressing these problems requires a thoughtful reassessment of the game's rules and mechanics. By implementing modifications that prioritize fairness, remove potentially controversial elements, and enhance player engagement through balanced gameplay opportunities, we can create a more inclusive and enjoyable experience. This approach not only aligns with the game's objective of providing fun and educational content for children but also upholds the values of equitable play and positive interaction among participants.

5 Problem Description

5.1 Set up

- Place the small game board on the large game board and fasten it with the four attaching cylinders as you see in the illustration.

Figure 11: Setting up the board

- Sort the worm sections according to colors and put them on the game board.

Figure 12: Provided worm sections

- Each player chooses and takes a worm head with their corresponding daisy and strawberry.

Figure 13: Player will need to have one of each to start the game

- The players burry their worm head until it disappears into the corresponding tunnel

5.2 Formal Description

The game starts with the youngest "burrower" (player) and other players will follow clockwise.
The player's turn consists of 3 actions:

- The player will start by rolling the dice for the corresponding worm sections

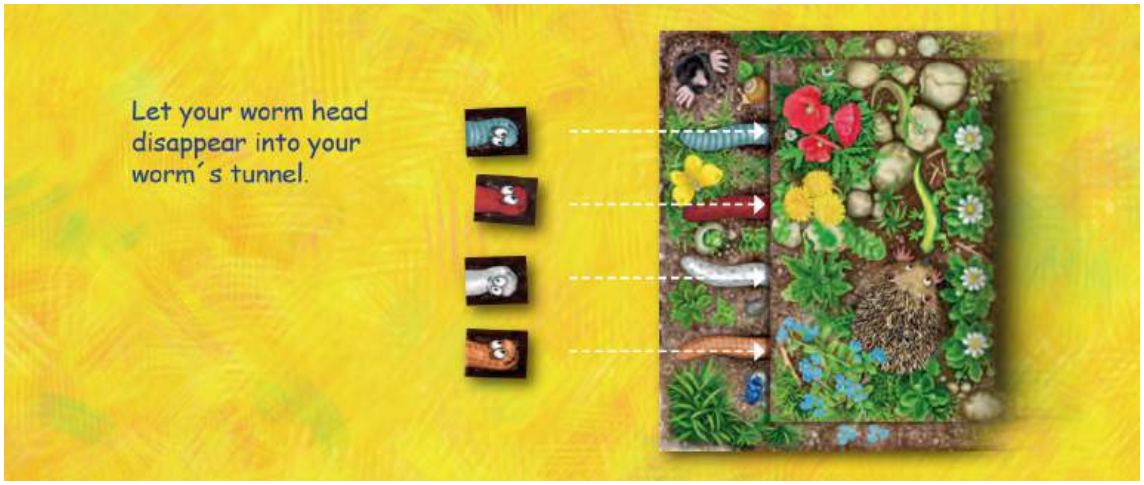


Figure 14: Starting the game

- The player will take the corresponding worm section and shove it into their worm tunnel until the piece disappears inside

Figure 15: Example of a player normal turn

- The player at any turn can estimate which worm will be the first to come up at the daises/strawberry patch and when they are confident with their choices, they can lay a daisy/strawberry on one of the daises/strawberry on the gameboard exactly on the worm tunnel that the player thinks will show up first

Figure 16: Example of how the player would estimate using a daisy

5.2.1 Additional rules

- When the first worm appears right there at the daisies/strawberry, the player can feed their worm with the daisy if they place it correctly

Figure 17: Example of how the player would estimate using a daisy

- Incorrectly played daisies or strawberries are removed from the game.
- When the first worm appears right there at the daisies/strawberry, the player cannot place their bet anymore on the corresponding patch
- The player can place the daisy first and only when the first worm appears at the daises then the player can now bet on the strawberry
- The player can place their daisies/strawberries even if others' daisies/strawberries are already lying there.
- If no worm section is left in this color, the player can pick any other section to feed their worms.
- The player can change their pick of the daisies/strawberries on their turn but they cannot once the worm shows their head at the patch.

5.2.2 Winning conditions

The worm that is first to stretch his head out of the soil at the compost heap wins the game.

6 Related Work

The game developed in this report shares similarities with other turn-based games that rely on player actions and random events. The use of a dice roll mechanism to determine turns is a common element found in many board and card games. Similarly, the concept of moving worms around a grid and using betting rounds to gain points is reminiscent of games like Snakes and Ladders[8].

Figure 18: Snakes and Ladders game

The game uses a form of fractal geometry to generate the grid for the game board. Fractals are self-similar patterns that occur at different scales, which makes them ideal for creating complex and visually appealing designs. In this case, the fractal pattern is used to create the grid layout for the game board.

The game also employs a simple form of AI, in the form of the dice roll mechanism. The dice roll determines the order of turns and introduces an element of chance into the game. This is a common feature in many board games, adding an extra layer of unpredictability and excitement.

While the game does not explicitly use any advanced algorithms such as genetic algorithms, PSO (Particle Swarm Optimization), or ACO (Ant Colony Optimization), the principles of these algorithms could potentially be applied to improve aspects of the game. For instance, genetic algorithms could be used to evolve the strategies of the worms over time, or PSO could be used to optimize the placement of the worms on the game board.

7 Proposed Approaches

7.1 Input/Output Format

Our game's input/output (I/O) formats are designed to be straightforward and user-friendly, ensuring that players can easily interact with the game and understand the feedback provided by the game's interface.

Input: The game accepts user input mainly through mouse clicks. Players can interact with the game by clicking on various elements such as dice, boxes, and buttons. The *handleInput* method within each state class captures and processes these inputs.

```
public void handleInput(float dt) {
    if (Gdx.input.isButtonJustPressed(Input.Buttons.LEFT)) {
        // Process click input
    }
}
```

Output: The game provides visual output on the screen, such as the rendering of game entities, display of current player turns, and notifications of bonus moves. This output is managed within the *render* method of each state class, which updates the game's visual elements for each frame. In the example below draws the Play button to the screen in the position x = 900 and y = 350:

```
public void render(float delta) {
        // Update and draw game elements
        game.batch.begin();
        game.batch.draw(playButton,900, 350);
        // Draw other game elements
        game.batch.end();
}
```

7.2 Algorithms in Pseudocode

The functionality and dynamism of our game are driven by several core algorithms, each meticulously designed to govern specific aspects of gameplay. These algorithms are foundational to the game's mechanics, influencing everything from character movement to the progression of game states and the interactive betting system. Presented below are pseudocode descriptions of these key algorithms, offering insights into the logical framework that underpins our game's operation.

7.2.1 Worm Movement Algorithm

The Worm Movement Algorithm is responsible for updating the position and points of each worm based on the outcome of dice rolls. It ensures dynamic gameplay, directly influencing the game's competitive elements.

```
function move(worm, steps):
    worm.x += steps * 20
    worm.point += steps
    if worm.point exceeds threshold for state change:
        updateGameState()
```

7.2.2 GameState Update Algorithm

This algorithm plays a critical role in transitioning the game through various states based on the worms' points, adding layers of complexity and engagement to the gameplay.

```
function updateGameState():
    if all worms have points > 20:
        change to GameState2
    if any worm has points > 39:
        change to GameState3
    if any worm has points >= 55:
        end game and declare winner
```

7.2.3 Betting System Update Algorithm

The Betting System Update Algorithm enhances the game's strategic depth by allowing players to place bets on worms, awarding bonus moves based on these bets.

```
function updateBetting(ticks):
    for each tick in ticks:
        if tick is clicked and status is false:
            reset all ticks in the same column to false
            set clicked tick status to true
            assign bonus move to corresponding worm
```

7.2.4 Dice Roll Algorithm

The Dice Roll Algorithm introduces an element of chance into the game, dictating the movement of worms and influencing the game's outcome.

```
function rollDice():
    number = random number between 1 and 6
    play dice roll sound
    return number
```

7.2.5 GameStateManager Class

The GameStateManager class is crucial for managing the game's various screens and states, facilitating a smooth transition and enhancing user experience.

```
// Create a GameStateManager object
function GameStateManager(game):
    Set game property to passed game argument
```

```
    Initialize screens as an empty Stack
    Load background music file and set properties
    Start playing the background music

// Set the current screen and dispose of the previous one if any
function set(screen):
    If screens stack is not empty then
        Pop the top screen from the stack and call its dispose method
    Push the new screen to the top of the screens stack
    Set the current screen of the game to the new screen

// Add a new screen to the top of the screens stack
function push(screen):
    Push the new screen to the top of the screens stack
    Set the current screen of the game to the new screen

// Remove the top screen from the screens stack and dispose of it
function pop():
    Pop the top screen from the screens stack and store it in a variable called
        screen
    Call the dispose method on the popped screen
    If screens stack is not empty then
        Set the current screen of the game to the next screen at the top of the
            screens stack
```

7.2.6 Switching Game State Algorithm

This algorithm outlines the conditions and logic for transitioning between different game states, ensuring a coherent flow and progression within the game.

```
// Pseudocode for switching game state 1 to 2

// Check if the point of the fourth worm is not zero
if the point of worm number 3 is not equal to 0 then
    Set isMainState1 to true

// Loop through each worm
for each worm in the array of worms from index 0 to 3 do
    // Check if the point of the current worm is greater than 20
    if the point of current worm is greater than 20 then
        Set isMainState1 to false
        Set isMainState2 to true

        // Check if isMoveBox1 is false
        if isMoveBox1 is false then
            // Loop through each tick
            for each tick in the array of ticks from index 0 to 3 do
                // Check if the status of the current tick is true
                if the status of current tick is true then
                    Move the current worm by 3 steps
```

```
                Set the bonus of the current worm to true
        Set isMoveBox1 to true
```

Each algorithm described here is integral to the unique gameplay experience we have crafted. They collectively contribute to the game's logic and flow, dictating the behavior of in-game elements and the transition between game states. Through these algorithms, we have aimed to create a balanced, engaging, and dynamic game that challenges players and keeps them engaged.

8 Implementation Details

The implementation of our game was carried out with careful attention to detail in various aspects such as application structure, GUI details, UML diagrams, used libraries, and code snippets. This section aims to provide insight into the technical intricacies of our game's development process.

8.1 Application Structure

Our game leverages the **libGDX** framework, which provides a robust structure for managing game states and transitions. The architecture is modular, with separate classes for different game states such as *MenuState*, *PlayState*, *SettingState* and *EndState*. Each state represents a unique phase of the game, such as the main menu, gameplay, settings, and the game over screen, respectively.

The game is structured around a **State Management System**, which is a common design pattern in game development that allows for clean transitions between different screens or states of the game (such as the menu, gameplay, and end screens). Our implementation utilizes the *GameStateManager* class to handle these transitions.

GameStateManager: This class manages a stack of Screen objects, enabling the game to push new states onto the stack, pop states off, and replace the current state with a new one. It simplifies navigation between different parts of the game.

8.2 GUI Details

The game's GUI is designed to be intuitive and user-friendly. It includes buttons for navigation, interactive elements like the dice that players can roll, and visual indicators such as the worms' positions on the screen and the current player's turn. The GUI also features notifications for bonus moves and betting sets, enhancing the player's strategic involvement in the game. It is rendered using libGDX's powerful graphics capabilities, which allow for efficient drawing and updating of visual elements.

8.3 UML Diagrams

The following relationships between classes can be inferred and thus represented with appropriate lines in a UML diagram

The class diagram depicts the relationships between classes in a game. The diagram shows that the GameStateManager class is responsible for managing the game states. It has a **gsm** field that references itself, a screens field that is a Stack of Screen objects, a **bgm** field that references a Music object, and a **bmVolume** field that is a float. The GameStateManager class has a constructor and methods for **setting**, **pushing**, **popping**, and **disposing all of the screens**.

The MenuState, PlayState, EndState, and SettingState classes are all subclasses of the Screen class. They all have a **gsm** field that references the GameStateManager object, a game field that references a Worm object, and a constructor. The MenuState class has a handleInput() method that handles user input, a update() method that updates the state of the game, a render() method that renders the state of the game, a resize() method that resizes the game, and a dispose() method that disposes of the game. The PlayState class has the same methods as the MenuState class. The EndState class has a render() method that renders a message that the game is over. The SettingState class has a handleInput() method that handles user input for changing the game settings, an update() method that updates the game settings, a render() method that renders the game settings screen, and a resize() method that resizes the game settings screen.

The Dice class represents a dice. It has a **batch** field that references a SpriteBatch object, an **x** field that is a float, a **y** field that is a float, a **turn** field that is an int, a **currentNumber** field that is an int, and a constructor. The Dice class has a getTurn() method that returns the turn of the dice, a roll() method that rolls the dice, and a update() method that updates the state of the dice.

The Worm class represents a worm. It has a **texturePath** field that is a String, a batch field that references a SpriteBatch object, an **x** field that is a float, a **y** field that is a float, a **id** field that is a String, a **point** field that is an int, and a **bonus** field that is a boolean. The Worm class has a constructor, a getPoint() method that returns the point of the worm, a move() method that moves the worm, a update() method that updates the state of the worm, and a render() method that renders the worm.

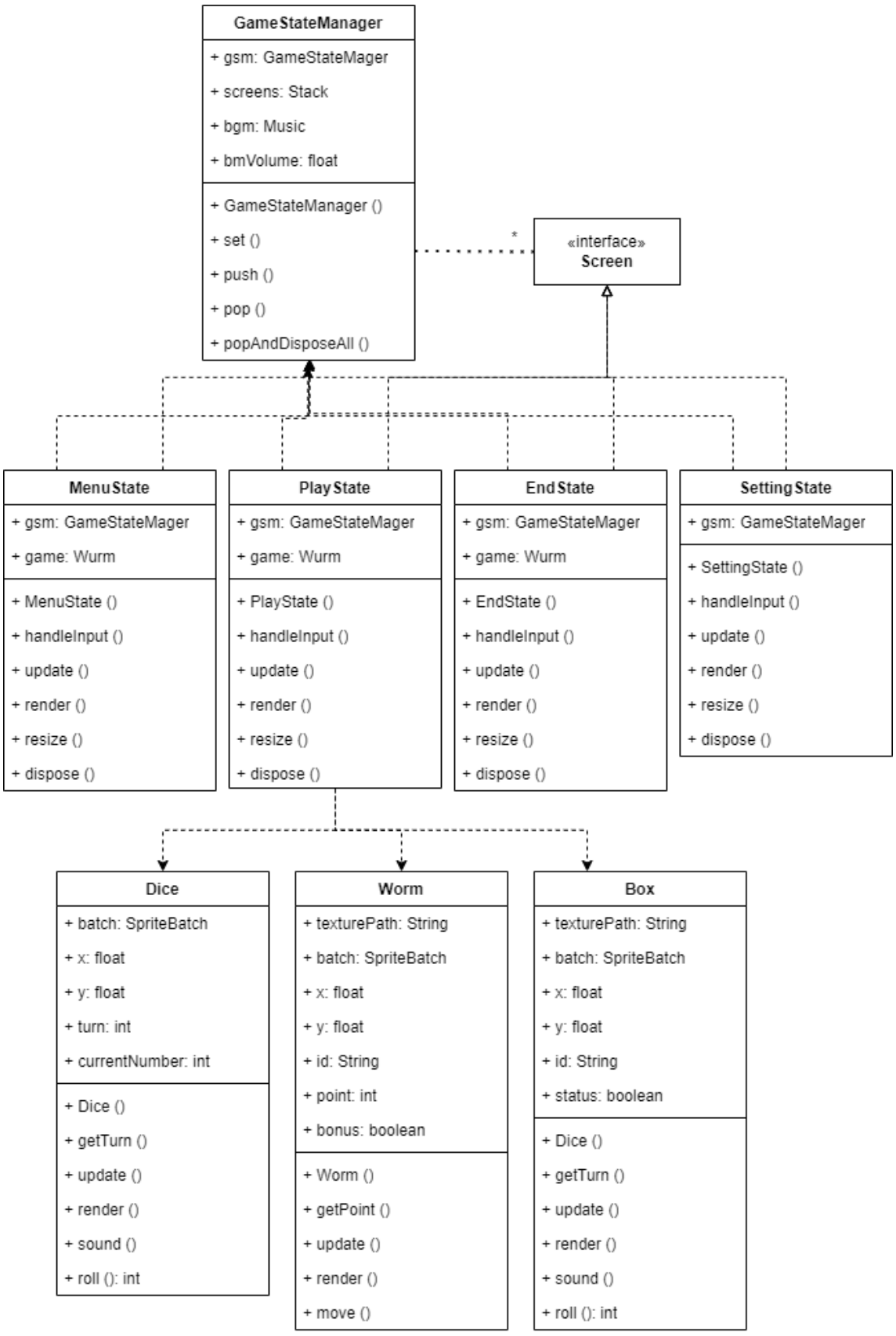


Figure 19: **UML project diagram**

8.4 Code Snippets

Here are some code snippets that illustrate key aspects of the implementation:

8.4.1 Worm Movement

This method moves the worm based on the number of steps calculated from the dice roll.

```
public void move(int step){
    x = x + step * 20;
    point += step;
}
```

8.4.2 Handling Betting Logic

This input-handling snippet allows players to change their bets. It checks if a bet box is clicked and updates the status array accordingly.

```
public void handleInput(float dt) {
    if (Gdx.input.isButtonJustPressed(Input.Buttons.LEFT)) {
        float mousex = Gdx.input.getX();
        float mousey = Gdx.graphics.getHeight() - Gdx.input.getY();
        for (int i = 0; i < 4; i++){
            for (int j = 0; j < 4; j++){
                if (ticks[i][j].check(mousex,mousey)){
                    for (int k = 0; k < 4; k++){
                        ticks[k][j].status = false;
                    }
                    ticks[i][j].status = true;
                }
            }
        }
    }
```

8.4.3 Moves Based Random Dice

This function take random number in range 1 to 6 as dice simulation

```
public int roll(){
    return (int)(Math.random() * 6) % 6;
}
```

This dice's update function take the result of roll() function pass to worm's move when users click into the dice area on the screen.

```
public void update(){
        if (Gdx.input.isButtonJustPressed(Input.Buttons.LEFT)) {
            float mouse_x = Gdx.input.getX();
            float mouse_y = Gdx.graphics.getHeight() - Gdx.input.getY();
            System.out.println(mouse_x + "␣" + mouse_y);
```

```
            if ((mouse_x >= x && mouse_y >= y) && ((mouse_x <= x+texture[0].
                getWidth()) && (mouse_y <= y + texture[0].getHeight())))
            {
                currentNumber = roll() + 1;
                sound();
                PlayState.worms[turn].move(currentNumber);
                turn = (turn + 1) % 4;
            }
        }
    }
```

8.4.4 GameStateManager Push and Pop Mechanism

The *push* method adds a new screen to the stack and sets it as the current screen.

```
public void push(com.badlogic.gdx.Screen screen) {
    screens.push(screen);
    game.setScreen(screen);
}
```

The *pop* method removes a current screen from the stack and sets the previous screen as the current screen.

```
public void pop() {
    com.badlogic.gdx.Screen screen = screens.pop();
    screen.dispose();
    if (!screens.isEmpty()) {
        game.setScreen(screens.peek());
    }
}
```

8.4.5 Handling Input in PlayState

This method receives click input from the player with the corresponding position chosen from the screen

```
public void handleInput(float dt) {
    if (Gdx.input.isButtonJustPressed(Input.Buttons.LEFT)) {
        // Handle left click input
    }
}
```

8.4.6 Switch between game state

This code check whenever the last worm do its turn, the game will change from game state 1 to game state 2.

```
if (worms[3].getPoint() != 0){
    isMainState1 = true;
}
```

```
for (int i = 0; i < 4; i++){
    if (worms[i].getPoint() > 20){
        isMainState1 = false;
        isMainState2 = true;
        if (!isMoveBox1){
            for (int j = 0; j < 4; j++){
                //update bonus
                if (ticks[i][j].status){
                    worms[j].move(3);
                    worms[j].bonus = true;
                }
            }
            for (int j = 0; j < 4; j ++){
                for (int k = 0 ; k < 4; k++) {
                    ticks[j][k].status = false;
                    ticks[j][k].x += 380;
                }
            }
            isMoveBox1 = true;
        }
    }
}
```

This code check whenever which worm have their points greater than 39 points, the game will change from game state 2 to game state 3.

```
for (int i = 0; i < 4; i++){
    if (worms[i].getPoint() > 39){
        isMainState2 = false;
        isMainState3 = true;
        if (!isMoveBox2){
            bonusMove = "";
            for (int j = 0; j < 4; j ++){
                if (ticks[i][j].status){
                    worms[j].move(3);
                    worms[j].bonus = true;
                }
            }
            isMoveBox2 = true;
        }
        for (int k=0; k<4; k++){
            Wurm.batch.draw(sewage1, 840 + 20, 150*k);
        }
    }
}
```

8.4.7 Rendering GUI Elements in MenuState

This method renders images to screen

```
@Override
public void render(float delta) {
    // Clear screen and set camera view
    game.batch.begin();
    // Draw Play, Setting, Exit buttons
    game.batch.end();
}
```

8.4.8 Music control

This code set background music loop during entire game.

```
bgm = Gdx.audio.newMusic(Gdx.files.internal("music/backgroundSound.mp3"));
    bgm.setLooping(true);
    bgm.setVolume(bgmVolume);
    bgm.play();
```

When switching to the end of the game (EndState) the background music will stop and play the winner sound of the game.

```
GameStateManager.bgm.pause();
wm = Gdx.audio.newMusic(Gdx.files.internal("music/winnerSound.mp3");
wm.setVolume(0.1F * wmVolume);
wm.play();
```

When player want to play another game, winner sound will stop and the background music sound will play.

```
if (x >= 870 && x <= 870 + 220 &&
        y >= 350 && y <= 350 + 70) {
    if (wm.isPlaying()) {
        wm.stop();
    }
    GameStateManager.bgm.play();
}
```

9 Experimental Results, Statistical Tests, Running Scenarios

9.1 Simulations

This section describes simulations run to test the game mechanics.

- Start the game (Menu Screen)

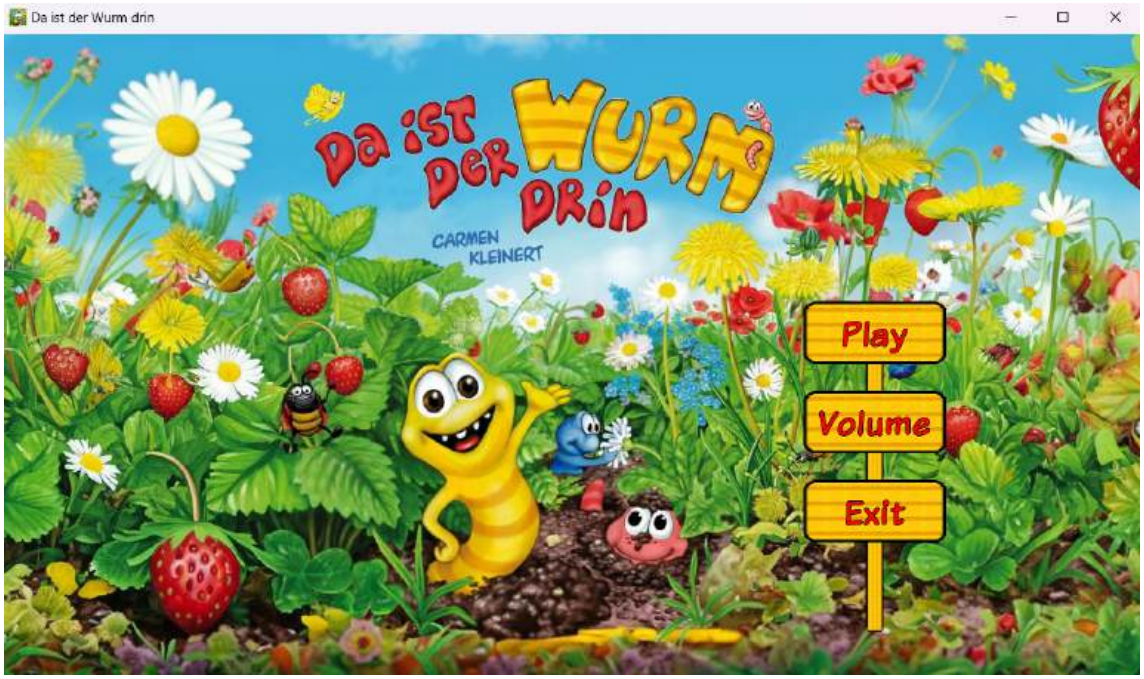
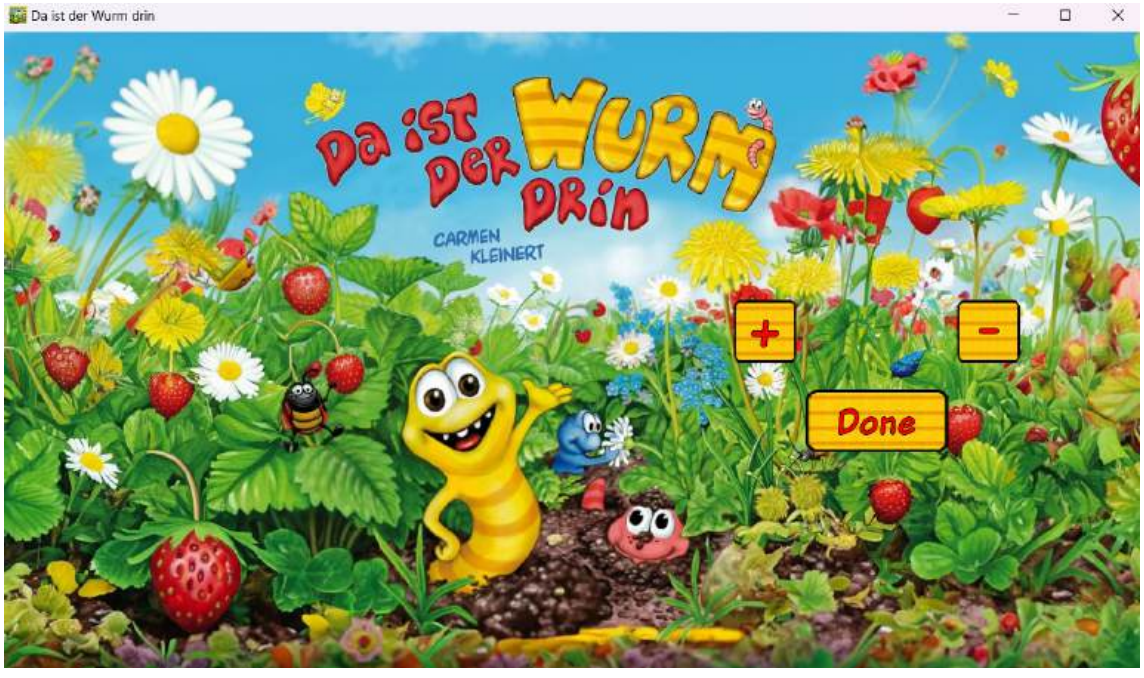
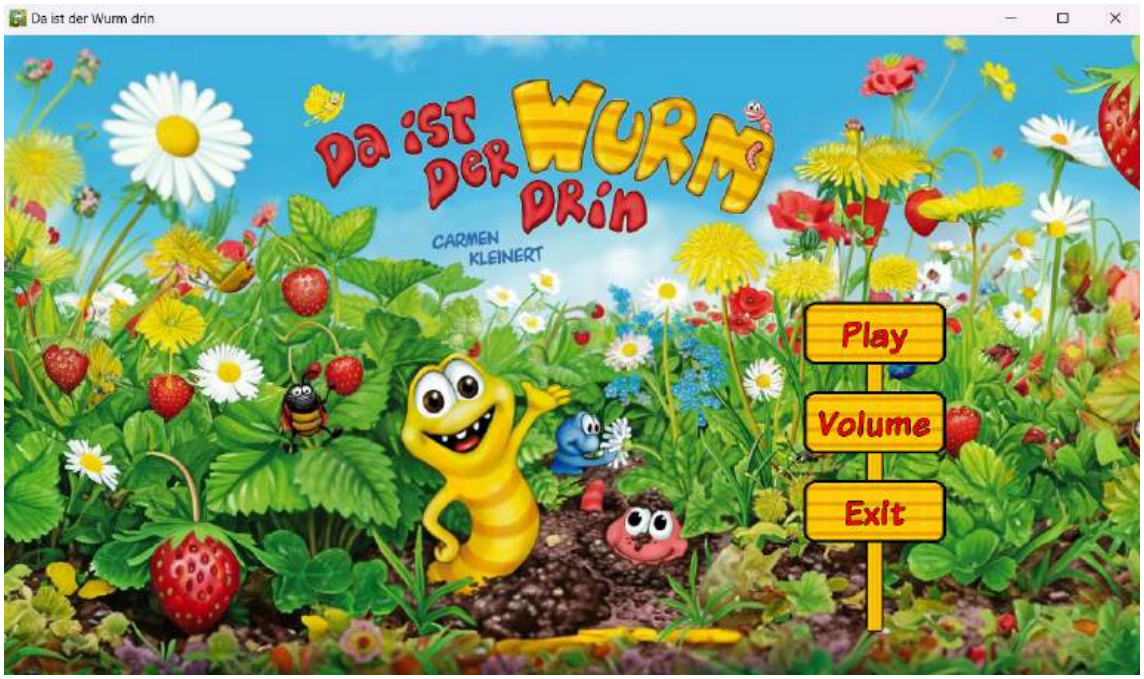


Figure 20: Menu Screen

- Display Setting Screen when user click "Volume" from Menu screen. This screen is control game's volume sound background.



- Display back to Menu screen when user click “Done” from Setting Screen

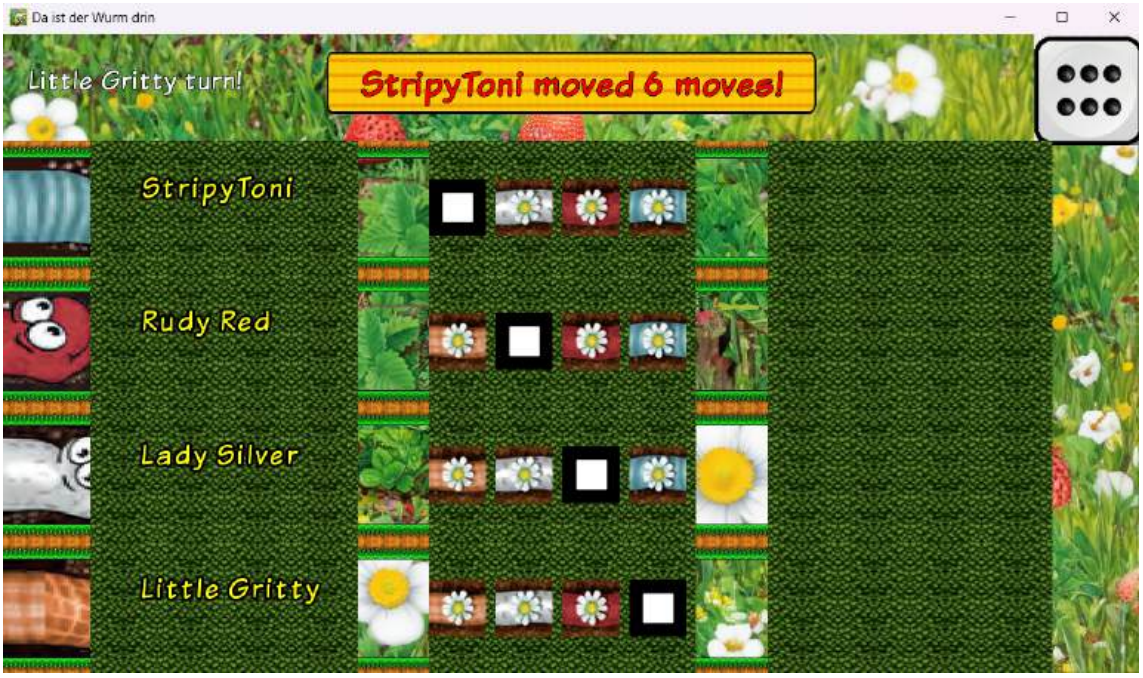


- Display Play screen when user click “Play” from Menu screen

- The game will start by clicking the dice on the top right of the screen. The worm Little Gritty will move first by default (e.g., 4 moves corresponding to the dice after clicking and display to the notification box)

- Display betting when all worms do it's first turn.

- Click to the flowers to bet.



- Please note that player can change their betting during any of their turns. This example illustrate Little Gritty bet to itself if it go to eat flowers first. Lady Silver bet to Rudy Red and StripyToni & Rudy Red bet to Lady Silver

- Little Gritty eat flowers first and receive 3 bonus move. The game switch to game state 2.

- In the game state 2, all worms bet to Little Gritty.

- Since Little Gritty eat strawberries first, all worms receive 3 bonus moves from successful betting. The game switch to game state 3.

- Since Little Gritty go to the goal first, the screen display winner notification in the box. The game end.

- The screen will automatically switch to End screen after 3 seconds. In this screen, if players want to play again, click “New game”, otherwise click “Exit”

9.2 Tables

9.2.1 Game State Transition Table

Table that shows the conditions required for transitioning between different game states.

State	Condition
Game State 1	$0 \leq point \leq 20$
Game State 2	$21 \leq point \leq 39$
Game State 3	$40 \leq point \leq 54$
Winner	$point \geq 55$

Table 2: Game Condition

9.2.2 Entity Update Table

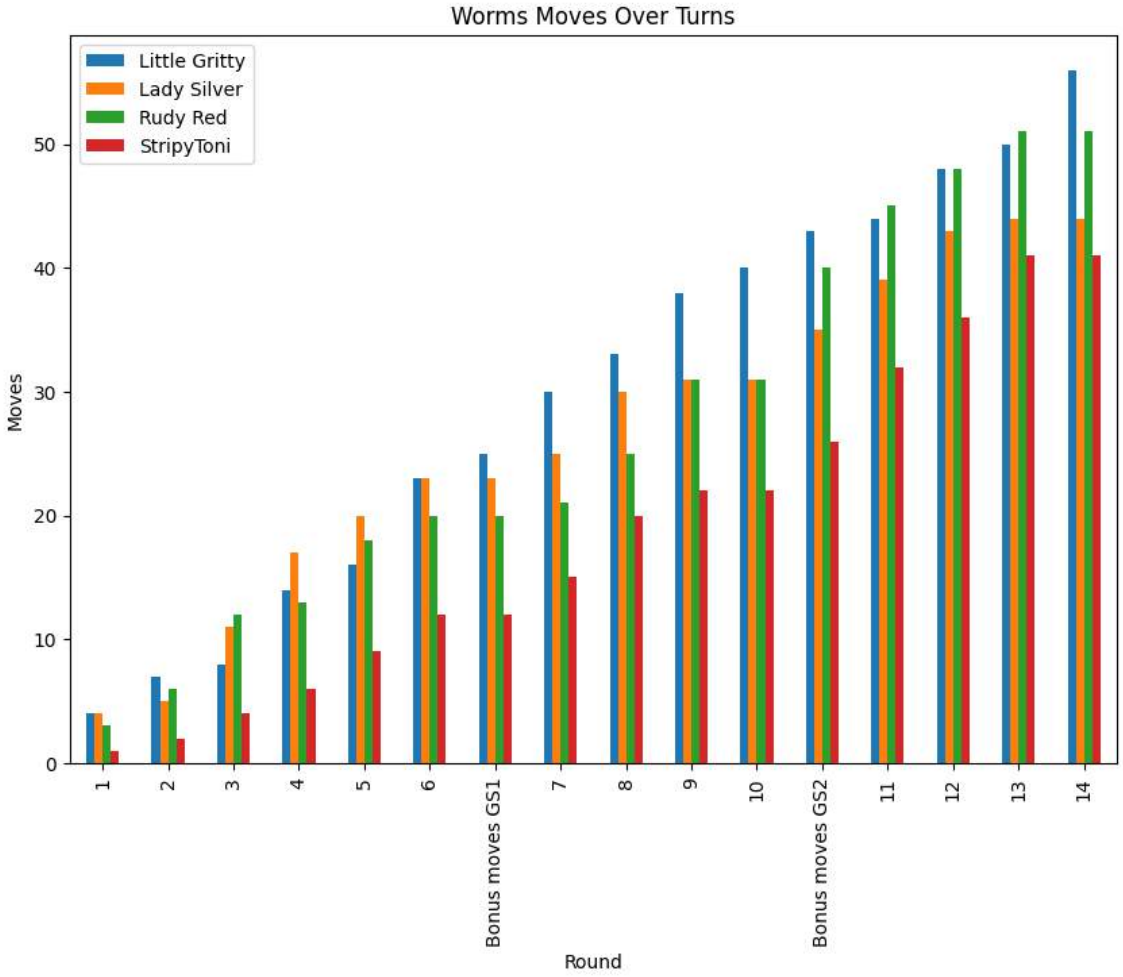
A table that shows how different entities in the game are updated over time from the simulations:

Round	Little Gritty	Lady Silver	Rudy Red	StripyToni
1	4	4	3	1
2	7	5	6	2
3	8	11	12	4
4	14	17	13	6
5	16	20	18	9
6	23	23	20	12
Bonus moves game state 1	25	23	20	12
7	30	25	21	15
8	33	30	25	20
9	38	31	31	22
10	40	31	31	22
Bonus moves game state 2	43	35	40	26
11	44	39	45	32
12	48	43	48	36
13	50	44	51	41
Winner	56	44	51	41

Table 3: Recorded simulation data

9.3 Charts

This is a bar chart showing visualize how the score of each worm changes over time in the simulation:



Below is our code to plot the bar chat in Python.

```
import pandas as pd
import matplotlib.pyplot as plt

# Data
data = {
    'Round': [1, 2, 3, 4, 5, 6, "Bonus moves GS1", 7, 8, 9, 10, "Bonus moves GS2",
        11, 12, 13, 14],
    'Little Gritty':[4, 7, 8, 14, 16, 23, 25, 30, 33, 38, 40, 43, 44, 48, 50, 56],
    'Lady Silver':[4, 5, 11, 17, 20, 23, 23, 25, 30, 31, 31, 35, 39, 43, 44, 44],
    'Rudy Red':[3, 6, 12, 13, 18, 20, 20, 21, 25, 31, 31, 40, 45, 48, 51, 51],
    'StripyToni':[1, 2, 4, 6, 9, 12, 12, 15, 20, 22, 22, 26, 32, 36, 41, 41]
}

# Create DataFrame
df = pd.DataFrame(data)

# Set the index to 'Round'
df.set_index('Round', inplace=True)

# Plot
df.plot(kind='bar', figsize=(10, 7))
```

```
# Set labels and title
plt.xlabel('Round')
plt.ylabel('Moves')
plt.title('Worms Moves Over Turns')

# Show the plot
plt.show()
```

9.4 Evaluations

- **Game Mechanic Evaluation**

  The betting mechanism is a pivotal aspect of the game, shaping the overall dynamics and progression of the game. This system allows players, even those with fewer points initially, to improve their scores through strategic betting rounds. This approach provides an opportunity for skill and strategy to come into play, making the game more engaging and enjoyable.

  However, it's important to balance this aspect of the game. While allowing players with lower points to increase their scores through betting can lead to a more level playing field, it must not result in a situation where players who start with an advantage can maintain that advantage indefinitely. Therefore, careful consideration needs to be given to the rules governing betting and scoring to ensure fairness and balance.

- **User Interface Evaluation**

  The user interface of the game is designed to be intuitive and user-friendly, facilitating seamless interaction between the players and the game. This design choice enhances the overall user experience, making the game more accessible and appealing to a wider audience.

  However, while the UI is user-friendly, it's equally important that it remains engaging and interesting. A complex or boring UI can quickly deter players, regardless of how intuitive it is. Therefore, while keeping the UI simple and clean, it's also crucial to incorporate elements of visual appeal and intrigue to keep players engaged.

10 Conclusions and Future Work

10.1 How was the Team Work

- Communication: There were a few problems at the start as we did not share our perspective about the game except for the basics which led to everyone working with a different version of the game in mind. This may have been set up a few days back due to just being bull-headed into coding and focusing on our part rather than sharing our thoughts and vision about the game with each other. However, this also gave us insight into how the game should work and what it needs during our first weekly meeting. On the other hand, we also elect a team leader to keep track of our overall progress and create a chat group for when we have any ideas or questions.

- Task Allocation and Management: The task allocation was unbalanced as one member already had experience and prior knowledge of using Java to develop games while the rest ran into some troubles and many difficulties however we tried to keep it balanced by self-studying basic concepts and got supported by the member with better experience in that area. However, due to time constraints and to make the best version of the game following our vision, the experienced member responded to more tasks and higher difficulty tasks. Procrastination was another issue that affected productivity.

10.2 Key Lessons Learned

The journey of our game development was not just about creating an engaging product but also about the invaluable lessons we learned along the way. These lessons have significantly shaped our approach to software development, emphasizing the importance of certain practices that ensure efficiency, reliability, and continual improvement. Here, we detail some of the critical insights gained during our project's lifecycle.

10.2.1 Importance of Saving Backup Versions of Code

One of the foremost lessons we learned is the critical importance of saving backup versions of our code. This practice was not merely about having a fallback option; it was about creating a comprehensive history of our development process. By maintaining regular backups, we were able to track the evolution of our codebase, understand the implications of various changes, and, most importantly, recover quickly from unforeseen issues or errors. This approach underscored the value of version control systems, which became an integral part of our workflow, enabling us to navigate the complexities of game development with greater confidence and control.

10.2.2 Need for Well-Structured Code

Another key lesson was the undeniable need for well-structured code. Early in the development process, we recognized the importance of adopting proper naming conventions for variables and ensuring appropriate scoping. These practices were not just about maintaining a clean codebase;

they were crucial for the maintainability and readability of our code. By adhering to these standards, we facilitated easier code reviews, streamlined collaboration among team members, and significantly reduced the time spent on debugging and refactoring. This experience reinforced the notion that well-structured code is the backbone of any successful software project, paving the way for scalability and adaptability.

10.2.3 Documentation of the Development Process

Documenting the development process emerged as another vital lesson from our project. Keeping detailed notes on various stages of the project, including idea generation, debugging efforts, and brainstorming sessions, proved to be instrumental in tracking our progress and pinpointing areas for improvement. This documentation served not only as a record of our journey but also as a learning tool, allowing us to reflect on our decisions, evaluate our strategies, and make informed adjustments. The practice of documenting our process fostered a culture of transparency and continuous learning within our team, highlighting the importance of reflective practice in the realm of software development.

In conclusion, these lessons learned are not merely reflections on our past experiences; they are guiding principles for our future endeavors. The insights gained from the challenges we faced have equipped us with a deeper understanding of the intricacies of game development, informing our approach to future projects. By sharing these lessons, we hope to contribute to the broader community of developers, emphasizing the continuous nature of learning in the field of software development.

10.3 Ideas for Future Development

As we reflect on the journey of our game's development and its current state, we recognize that the path of innovation and improvement is endless. Looking ahead, there are several avenues for potential enhancements and the introduction of new features that could significantly elevate the gaming experience. This section outlines our vision for the future development of the game, focusing on usability, performance, and engagement. By exploring these ideas, we aim to lay the groundwork for a more dynamic, immersive, and enjoyable game.

10.3.1 Dynamic Switching Between Play State and Setting State

One of the key areas for improvement is the mechanism for switching between the Play State and the Setting State. Currently, the transition between these states is functional, yet there exists an opportunity to make this process more dynamic and fluid. Enhancing the usability and flexibility of state transitions could lead to a more seamless gaming experience, minimizing disruptions and maintaining player engagement. Implementing a more intuitive and responsive switching mechanism is envisaged to contribute significantly to the overall user experience, making navigation within the game more effortless and intuitive.

10.3.2 Adding Sound Effects for Successful Bets

Another exciting prospect for future development is the addition of sound effects to signify successful bets. Sound plays a pivotal role in creating an immersive gaming environment, and by incorporating auditory feedback for winning bets, we can add an extra layer of excitement and satisfaction for players. This feature would not only enrich the sensory experience but also provide immediate and gratifying acknowledgment of players' strategic successes, further enhancing the game's engagement factor.

10.3.3 Optimizing Code for Better Performance

The performance of the game is paramount to delivering a smooth and enjoyable experience. There is always room for optimizing the code to enhance efficiency and reduce lag times. By structuring the code for better performance, we can ensure that the game runs more smoothly across various devices, minimizing delays and improving responsiveness. This optimization process involves refining algorithms, reducing redundancy, and employing best practices in coding to achieve a more streamlined and efficient codebase. The end goal is to provide a seamless gaming experience that players can enjoy without the hindrance of performance issues.

10.3.4 Implementing Full-Screen Display

Lastly, allowing the game to run in full-screen mode is a potential enhancement that could offer players a more immersive gaming experience. Full-screen display can maximize the visual impact of the game's graphics, drawing players deeper into the game world. By providing an option for full-screen mode, we cater to players' preferences for a more engaging and enveloping experience, making the game more visually captivating and enjoyable.

By considering and implementing these suggestions, we aim to further improve and expand upon the game, enhancing its appeal and enjoyment for players. These future development ideas represent our commitment to evolving the game, ensuring it continues to engage and delight players with new features and optimizations. The journey of enhancement is ongoing, and we look forward to exploring these possibilities as we strive to create a more immersive and satisfying gaming experience.

References

[1] Kleinert, C. (2011). Da ist der Wurm drin. Zoch Verlag. Retrieved from https://www.zoch-verlag.com/zoch_de/kategorien/kinderspiele/da-ist-der-wurm-drin-601132100-de.html

[2] Cong-Nguyen Le, Thien-Huong Duong, and Minh-Anh Nguyen. (2024). Da ist der Wurm drin game project in Java. Retrieved from https://github.com/lcnguyencs/Da-ist-der-Wurm-drin

[3] libGDX. (2024). libGDX frameworks. Retrieved from https://libgdx.com/

[4] Thorbjørn Lindeijer. (2008-2021). Tiled. Retrieved from https://www.mapeditor.org/

[5] Adobe. (2023). Adobe Firely. Retrieved from https://firefly.adobe.com/

[6] AdhesiveWombat. (n.d.). 8-bit Music AdhesiveWombat - Night Shade NO COPYRIGHT. Retrieved from https://www.youtube.com/watchK?v=mRN_T6JkH-c

[7] Vigilante Typeface Corporation. (n.d.). Komika Text. Retrieved from https://www.dafont.com/de/komika-text.font

[8] CBC. (2024). Snakes and Ladders. Retrieved from https://www.cbc.ca/kids/games/play/snakes-and-ladders